

# PHASE 2 — QUANTUM COMPUTING & THE QUANTUM THREAT TO CRYPTOGRAPHY

**Project:** Enterprise Cryptographic Discovery & Analysis Tool (ECDAT)

**Problem Statement:** SIH26164

**Organization:** National Technical Research Organisation (NTRO)

**Theme:** Blockchain & Cybersecurity

**Phase:** 2 of 22

**Document Type:** Technical Research & Engineering Handbook

**Status:** Foundation Research

---

## Table of Contents

1. Purpose of This Phase
2. Why ECDAT Needs Quantum Computing Knowledge
3. Classical Computing vs Quantum Computing
4. What Is a Qubit?
5. Superposition
6. Measurement
7. Entanglement
8. Quantum Gates
9. Quantum Circuits
10. Quantum Algorithms
11. Why Quantum Computing Threatens Cryptography
12. Shor's Algorithm
13. Integer Factorization
14. Discrete Logarithm Problem
15. Impact on RSA
16. Impact on Diffie-Hellman
17. Impact on Elliptic Curve Cryptography
18. Impact on Digital Signatures
19. Grover's Algorithm
20. Impact on Symmetric Cryptography
21. Hash Functions Under Quantum Attack

22. Classical vs Quantum Security
  23. Cryptographically Relevant Quantum Computer (CRQC)
  24. Quantum Resource Requirements
  25. Error Correction
  26. Logical vs Physical Qubits
  27. Why Building a CRQC Is Difficult
  28. Quantum Threat Timeline
  29. Harvest Now, Decrypt Later
  30. Long-Lived Sensitive Data
  31. The "Migration Before the Quantum Computer" Problem
  32. Mosca's Inequality
  33. Quantum Risk in Enterprise Infrastructure
  34. Quantum Risk by Cryptographic Asset
  35. Quantum Risk by Data Lifetime
  36. Quantum Risk by Business Criticality
  37. Quantum Risk by Attack Surface
  38. Quantum Risk Categories
  39. Example Enterprise Scenario
  40. Example ECDAT Analysis
  41. What ECDAT Must Understand
  42. Quantum-Aware Cryptographic Discovery
  43. Quantum Risk Metadata
  44. Quantum Risk Scoring
  45. Quantum Risk Visualization
  46. Quantum Migration Prioritization
  47. Common Misconceptions
  48. Important Distinctions
  49. Engineering Implications
  50. Phase 2 Summary
  51. Phase 3 Preview
- 

## 1. Purpose of This Phase

Phase 1 established the foundations of modern cryptography.

We now need to answer the most important question behind the NTRO problem statement:

**Why does quantum computing create a need for a completely new approach to cryptographic inventory and risk assessment?**

ECDAT is fundamentally a **post-quantum readiness platform**.

Therefore, simply understanding RSA, ECC, AES, certificates, keys, and protocols is not enough.

The ECDAT engine must understand:

- which algorithms are vulnerable to quantum attacks,
- which algorithms remain comparatively resilient,
- what type of quantum attack applies,
- how much security is lost,
- how long the protected information needs to remain secure,
- how long migration will take,
- and whether an organization has enough time to migrate.

This phase develops the quantum-computing foundation required to make those decisions.

---

## 2. Why ECDAT Needs Quantum Computing Knowledge

Consider the following enterprise:

```
Enterprise
|
├─ 4,000 Applications
├─ 12,000 Containers
├─ 800 APIs
├─ 50,000 Certificates
├─ 20,000 Cryptographic Keys
├─ 3,000 Repositories
└─ 200 TB Sensitive Data
```

ECDAT scans the organization and discovers:

```
RSA-2048      → 8,400 occurrences
ECC P-256     → 5,700 occurrences
AES-256       → 11,000 occurrences
```

|          |                      |
|----------|----------------------|
| SHA-256  | → 14,000 occurrences |
| RSA-1024 | → 320 occurrences    |
| SHA-1    | → 2,100 occurrences  |

A conventional security scanner might flag RSA-1024 and SHA-1 because they are already considered weak or deprecated.

But ECDAT must ask a different question:

**What happens when a sufficiently capable quantum computer exists?**

That changes the analysis dramatically.

For example:

|                                                                                            |
|--------------------------------------------------------------------------------------------|
| <b>RSA-2048</b>                                                                            |
| Classical Risk:                                                                            |
| Low/Moderate depending on implementation                                                   |
| Quantum Risk:                                                                              |
| Critical                                                                                   |
| Reason:                                                                                    |
| Shor's algorithm can theoretically solve the underlying factorization problem efficiently. |

Therefore, ECDAT cannot merely identify algorithms.

It must understand their **future security properties**.

---

### 3. Classical Computing vs Quantum Computing

Classical computers process information using bits.

A classical bit has one of two states:

0

or

1

A quantum computer uses **qubits**.

A qubit is governed by quantum mechanics and can exist in a quantum state involving both computational basis states.

This distinction enables quantum algorithms that can solve certain mathematical problems dramatically more efficiently than known classical algorithms.

---

## 4. What Is a Qubit?

A classical bit is:

```
bit ∈ {0, 1}
```

A qubit can be represented as:

```
 $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ 
```

where:

- $|0\rangle$  is one basis state,
- $|1\rangle$  is another basis state,
- $\alpha$  and  $\beta$  are complex probability amplitudes.

The amplitudes satisfy:

```
 $|\alpha|^2 + |\beta|^2 = 1$ 
```

The probability of measuring:

```
 $0 = |\alpha|^2$ 
```

and:

```
 $1 = |\beta|^2$ 
```

This mathematical structure is the foundation of quantum computation.

---

## 5. Superposition

Superposition means that a qubit can exist in a combination of basis states before measurement.

Conceptually:

```
Classical bit:
```

```
0 OR 1
```

```
Quantum bit:
```

$$\alpha|0\rangle + \beta|1\rangle$$

This does **not** simply mean that a quantum computer is "trying every answer simultaneously."

That popular explanation is misleading.

The real power comes from designing quantum operations so that amplitudes associated with incorrect or unwanted outcomes interfere destructively while desired outcomes interfere constructively.

This phenomenon is called **quantum interference**.

---

## 6. Measurement

When a qubit is measured, the result is observed as a classical value.

For a single qubit:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

Measurement produces either:

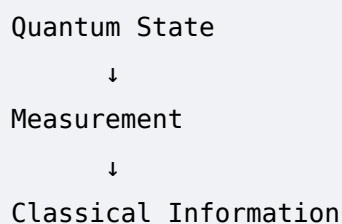
0

or:

1

with probabilities determined by the amplitudes.

Therefore:



This is important because quantum algorithms must carefully manipulate quantum states before measurement.

---

## 7. Entanglement

Entanglement is a quantum phenomenon where the state of multiple qubits cannot be described independently.

For example, two qubits may be represented by an entangled state such as:

$$(|00\rangle + |11\rangle) / \sqrt{2}$$

Measuring one qubit provides information about the corresponding state of the other.

Entanglement is one of the properties that allows quantum systems to exhibit correlations that have no straightforward classical equivalent.

---

## 8. Quantum Gates

Classical computers use logical gates such as:

- AND
- OR
- NOT
- XOR

Quantum computers use quantum gates.

Examples include:

- Pauli-X
- Pauli-Y
- Pauli-Z
- Hadamard
- Phase gates
- Controlled-NOT (CNOT)
- Rotation gates

The Hadamard gate, for example, can transform a computational basis state into a superposition.

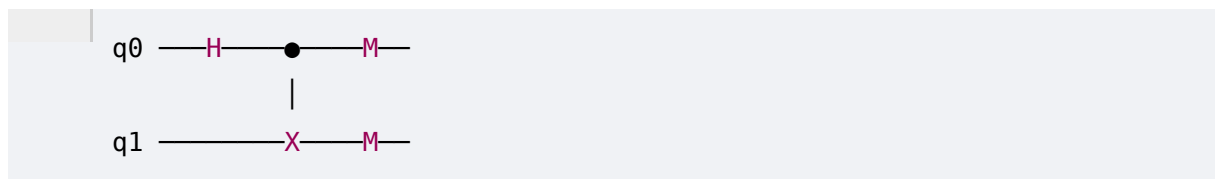
The CNOT gate is important for generating entanglement.

---

## 9. Quantum Circuits

Quantum algorithms are represented using quantum circuits.

A simplified representation:



Here:

- H = Hadamard gate
- ● = control
- X = Pauli-X
- M = measurement

Real quantum algorithms can involve thousands, millions, or more logical operations depending on the task and required error correction.

---

## 10. Quantum Algorithms

Quantum computing is not universally faster than classical computing.

Quantum advantage appears for specific computational problems.

Important algorithms include:

### Shor's Algorithm

Targets:

- Integer factorization
- Discrete logarithms

This is the primary reason public-key cryptography is threatened.

---

### Grover's Algorithm

Provides a quadratic speedup for generic unstructured search.

This affects:

- Symmetric cryptography
- Hash-based security assumptions

The impact is fundamentally different from Shor's algorithm.

---

## 11. Why Quantum Computing Threatens Cryptography

Most widely deployed public-key cryptography depends on mathematical problems believed to be computationally difficult for classical computers.

Examples:



RSA



Integer Factorization

Diffie-Hellman



Discrete Logarithm

ECC



Elliptic Curve Discrete Logarithm

A sufficiently capable quantum computer running Shor's algorithm could solve these problems efficiently enough to undermine the security assumptions behind these systems.

This is why RSA and ECC are considered **quantum-vulnerable**.

---

## 12. Shor's Algorithm

Shor's algorithm was introduced by Peter Shor in 1994.

It is one of the most important quantum algorithms in cybersecurity.

It provides an efficient quantum algorithm for integer factorization and discrete logarithms.

The significance is enormous.

Classically:

Large Integer



Factorization



Computationally Difficult

Quantum mechanically:

Large Integer



Shor's Algorithm



Efficient Quantum Procedure

The exact complexity and required resources depend on implementation and problem size, but the key cryptographic implication is clear:

The mathematical problems underlying RSA and many elliptic-curve and finite-field public-key systems do not provide long-term security against a sufficiently powerful fault-tolerant quantum computer.

## 13. Integer Factorization

RSA depends on the difficulty of factoring a large composite number.

Suppose:

$$N = p \times q$$

where  $p$  and  $q$  are large primes.

Multiplication is easy.

Recovering  $p$  and  $q$  from  $N$  is computationally difficult for appropriately chosen RSA parameters using known classical methods.

RSA security depends on this asymmetry.

Shor's algorithm changes the situation because a sufficiently capable quantum computer can solve integer factorization efficiently.

Therefore:

RSA

↓

Factorization

↓

Shor

↓

Quantum Vulnerability

## 14. Discrete Logarithm Problem

Several cryptographic systems rely on discrete logarithms.

For example, given:

$$g^x \bmod p = y$$

finding  $x$  from  $g$ ,  $p$ , and  $y$  is believed to be difficult for classical computers under suitable parameters.

This problem is used in systems such as:

- Diffie-Hellman
- DSA
- Elliptic Curve Cryptography

Shor's algorithm threatens these systems as well.

---

## 15. Impact on RSA

RSA is vulnerable to quantum computing because its security depends on integer factorization.

ECDAT should therefore classify RSA according to:

- Key size
- Usage
- Purpose
- Data protected
- Certificate lifetime
- Application criticality
- Migration complexity

For example:

### RSA-1024

- |
- ├─ Classical Security: Weak
- ├─ Quantum Security: Vulnerable
- └─ Priority: Critical

For:

### RSA-2048

- |
- ├─ Classical Security: Widely deployed
- ├─ Quantum Security: Vulnerable
- └─ Long-Term PQC Readiness: Poor

The distinction is important:

**"Not currently broken" does not mean "safe for long-lived information."**

---

## 16. Impact on Diffie-Hellman

Diffie-Hellman enables two parties to establish a shared secret over an insecure channel.

Its security relies on the discrete logarithm problem.

Quantum computers running Shor's algorithm threaten this assumption.

Therefore:

```
graph TD; A[Classical DH] --> B[Discrete Logarithm]; B --> C[Shor]; C --> D[Quantum Vulnerability]
```

Classical DH  
↓  
Discrete Logarithm  
↓  
Shor  
↓  
Quantum Vulnerability

The same concern applies to elliptic-curve variants such as ECDH.

---

## 17. Impact on Elliptic Curve Cryptography

ECC is attractive because it provides strong classical security with relatively small key sizes.

Common applications include:

- TLS
- SSH
- Digital signatures
- Mobile security
- Cryptographic tokens
- Blockchain systems

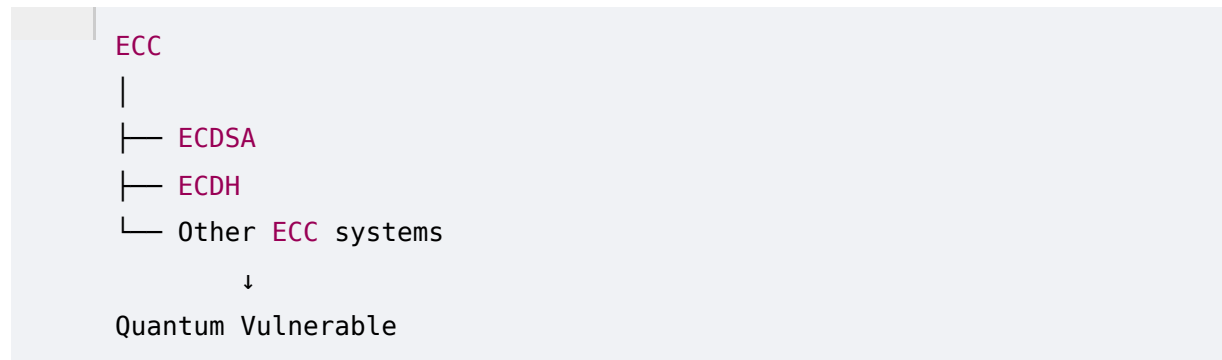
Examples include:

- ECDSA
- ECDH
- EdDSA

However, their security relies on elliptic-curve discrete logarithm problems.

Shor's algorithm threatens those problems.

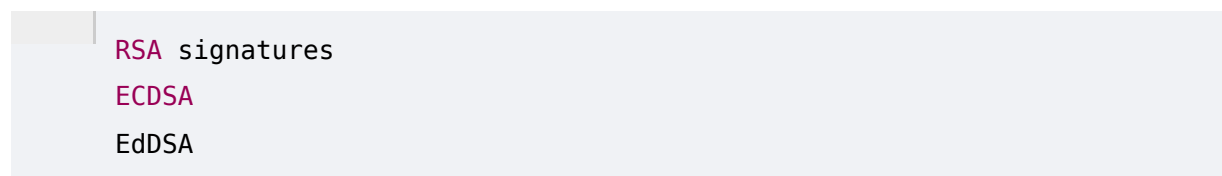
Therefore:



## 18. Impact on Digital Signatures

Digital signatures based on RSA or elliptic curves are also vulnerable to sufficiently capable quantum computers.

Examples:



This has major implications.

An attacker who obtains a private signing key could potentially:

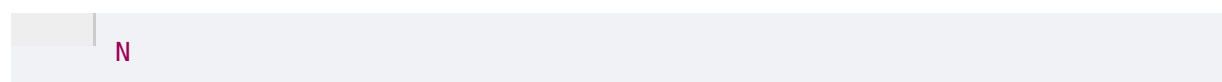
- Forge software signatures
- Impersonate services
- Sign malicious updates
- Forge documents
- Impersonate identities
- Compromise trust chains

Therefore, ECDAT must treat public-key signatures as a major migration category.

## 19. Grover's Algorithm

Grover's algorithm is fundamentally different.

Suppose a classical brute-force search over a space of size:



requires approximately:

$$O(N)$$

queries.

Grover's algorithm can reduce this to approximately:

$$O(\sqrt{N})$$

queries under the relevant oracle model.

This represents a quadratic speedup.

---

## 20. Impact on Symmetric Cryptography

The quantum impact on symmetric cryptography is not equivalent to the RSA/ECC situation.

For AES:

AES-128

has a nominal classical key space of:

$$2^{128}$$

A generic Grover-style quantum search could reduce the exponent conceptually to approximately:

$$2^{64}$$

under idealized assumptions.

Similarly:

AES-256

would have an idealized generic search cost around:

$$2^{128}$$

Thus AES-256 provides a significantly stronger security margin against generic quantum search.

This is one reason organizations evaluating long-term security may prefer larger symmetric security parameters.

---

## 21. Hash Functions Under Quantum Attack

Hash functions also face quantum considerations.

Grover's algorithm can affect preimage search.

However, the exact quantum security of hash functions depends on the security property being considered and the attack model.

For ECDAT, it is important to distinguish:

- Preimage resistance
- Second-preimage resistance
- Collision resistance

Collision attacks have different quantum complexity considerations from generic preimage search.

Therefore, ECDAT should **not** simply apply a universal "divide hash security by two" rule.

The risk engine must use algorithm-specific security knowledge.

---

## 22. Classical vs Quantum Security

A useful high-level model:

| Cryptographic Family | Classical Threat | Quantum Threat  |
|----------------------|------------------|-----------------|
| RSA                  | Factorization    | <b>Critical</b> |
| DH                   | Discrete log     | <b>Critical</b> |
| ECC                  | ECDLP            | <b>Critical</b> |

|         |                   |                                 |
|---------|-------------------|---------------------------------|
| ECDSA   | ECDLP             | <b>Critical</b>                 |
| AES-128 | Brute force       | Reduced margin                  |
| AES-256 | Brute force       | Stronger margin                 |
| SHA-256 | Classical attacks | Quantum security considerations |
| SHA-3   | Classical attacks | Quantum security considerations |

This table is simplified intentionally.

ECDAT's actual risk engine should maintain a richer algorithm knowledge base.

---

## 23. Cryptographically Relevant Quantum Computer (CRQC)

A CRQC is a sufficiently capable, fault-tolerant quantum computer that can perform attacks relevant to real-world cryptography.

This distinction is critical.

Today's experimental quantum computers are not automatically capable of breaking deployed RSA-2048 or P-256 systems.

There is a huge engineering gap between:

Quantum Computer Demonstration

and:

Cryptographically Relevant Quantum Computer



ECDAT should therefore avoid claims such as:

"Quantum computers can currently break RSA."

That is inaccurate.

The correct framing is:

"A sufficiently capable fault-tolerant quantum computer is expected to threaten widely deployed public-key cryptography."

---

## 24. Quantum Resource Requirements

Breaking cryptography requires much more than merely possessing qubits.

Relevant requirements include:

- Number of logical qubits
- Number of physical qubits
- Gate fidelity
- Error rates
- Circuit depth
- Error-correction overhead
- Coherence
- Connectivity
- Runtime
- Fault tolerance

This is one reason the transition to quantum-resistant cryptography is a **planning problem**, rather than an immediate panic scenario.

---

## 25. Error Correction

Quantum states are extremely sensitive to noise.

Errors can arise from:

- Environmental interactions
- Imperfect gates
- Measurement errors
- Decoherence

Quantum error correction attempts to protect logical information using multiple physical qubits.

Therefore:

```
Physical Qubits
    ↓
Error Correction
    ↓
Logical Qubits
    ↓
Fault-Tolerant Computation
```

A logical qubit may require many physical qubits.

The exact overhead depends heavily on hardware architecture, error rates, code choice, circuit requirements, and implementation.

---

## 26. Logical vs Physical Qubits

This distinction is essential when discussing quantum threats.

### Physical Qubit

An actual hardware element representing quantum information.

### Logical Qubit

An error-corrected quantum information unit constructed using multiple physical qubits.

Therefore:

```
1 Logical Qubit
    ≠
1 Physical Qubit
```

This matters because cryptanalytic estimates often involve **logical qubits and fault-tolerant operations**, not simply today's raw physical qubit counts.

---

## 27. Why Building a CRQC Is Difficult

A cryptographically relevant quantum computer requires solving enormous engineering challenges.

These include:

- Stable qubit technologies

- Low error rates
- Scalable control systems
- Fault-tolerant architectures
- Efficient error correction
- Large numbers of physical qubits
- High-fidelity gates
- Long computational circuits

Therefore, the exact arrival date of a CRQC is uncertain.

This uncertainty is precisely why organizations should not wait for a definitive announcement.

---

## 28. Quantum Threat Timeline

There is no universally accepted exact date for when a CRQC will exist.

ECDAT should therefore avoid implementing:

```
CRQC = 2038
```

as a hard-coded assumption.

Instead, it should support:

```
Estimated Quantum Threat Horizon
```

as a configurable parameter or scenario.

For example:

```
Scenario A:
```

```
Optimistic
```

```
Scenario B:
```

```
Moderate
```

```
Scenario C:
```

```
Aggressive
```

The organization can then evaluate migration risk under different assumptions.

---

## 29. Harvest Now, Decrypt Later

One of the most important concepts in post-quantum security is:

## Harvest Now, Decrypt Later (HNDL)

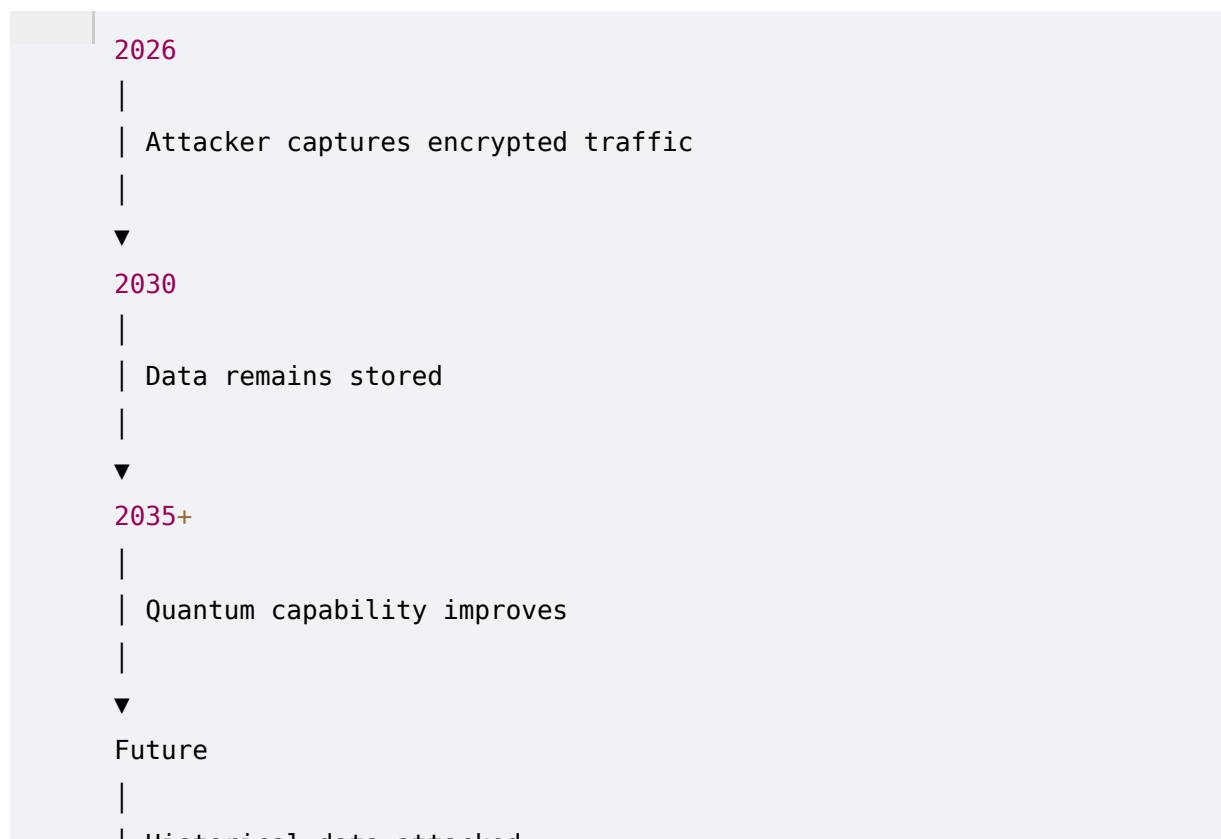
An attacker may capture encrypted traffic today.

They cannot decrypt it with current technology.

But they may store it.

Years later, if a sufficiently capable quantum computer becomes available, the attacker may attempt to decrypt historical traffic protected by vulnerable public-key mechanisms.

Conceptually:



This creates a security problem **before** the quantum computer exists.

## 30. Long-Lived Sensitive Data

Not all data has the same security lifetime.

Consider:

| Data | Required<br>Confident |
|------|-----------------------|
|------|-----------------------|

|                                    | ity                              |
|------------------------------------|----------------------------------|
| Temporar<br>y session<br>token     | Minutes                          |
| Shopping<br>cart                   | Hours                            |
| Business<br>transactio<br>n        | Years                            |
| Medical<br>record                  | Decades                          |
| Governme<br>nt<br>intelligenc<br>e | Decades                          |
| Defense<br>informatio<br>n         | Potentially<br>decades           |
| National-<br>security<br>material  | Potentially<br>extremely<br>long |

Therefore:

Quantum Risk  $\neq$  Algorithm Alone

It depends on:

Algorithm

+

Data Lifetime

- +  
Migration Time
- +  
Threat Horizon
- +  
Business Criticality

This concept becomes central to ECDAT.

---

## 31. The "Migration Before the Quantum Computer" Problem

Suppose an enterprise has:

- Data Lifetime = 20 years
- Migration Time = 5 years
- Estimated Quantum Threat Horizon = 15 years

Then:

$$20 + 5 > 15$$

The organization may need to act now.

Why?

Because the organization needs the data to remain secure longer than the estimated safe window allows.

This is why post-quantum migration is not something enterprises can simply begin after the first cryptographically relevant quantum computer appears.

---

## 32. Mosca's Inequality

This leads to the famous migration-risk model associated with Michele Mosca.

A simplified representation is:

$$X + Y > Z$$

where:

$X$  = Security Lifetime of Data

$Y$  = Migration Time

$Z$  = Estimated Time Until Cryptographically Relevant Quantum Computer

If:

$$X + Y > Z$$

the organization should treat the system as requiring urgent attention.

---

## Example

Suppose:

Sensitive Data Lifetime = 15 years

Migration Time = 4 years

Quantum Threat Horizon = 12 years

Then:

$$15 + 4 = 19$$

$$19 > 12$$

Therefore:

Quantum Migration Risk = HIGH

ECDAT should automatically calculate this relationship.

---

## 33. Quantum Risk in Enterprise Infrastructure

Quantum risk can appear in many locations.

### Application Layer

Application

|

└─ RSA

└─ ECC

- └─ TLS
- └─ JWT signing
- └─ Encryption APIs

## Infrastructure Layer

- Infrastructure
  - |
  - └─ VPN
  - └─ SSH
  - └─ TLS
  - └─ Certificates
  - └─ HSM

## Data Layer

- Data
  - |
  - └─ Database encryption
  - └─ Backup encryption
  - └─ Object storage
  - └─ Archive encryption

## Software Supply Chain

- Supply Chain
  - |
  - └─ Signed packages
  - └─ Container signatures
  - └─ Software certificates
  - └─ Dependency libraries

Every layer can contain quantum-vulnerable cryptographic dependencies.

---

## 34. Quantum Risk by Cryptographic Asset

ECDAT should classify discovered assets.

Example:

Asset:  
RSA-2048 Certificate



Type:  
Public-Key Encryption / Signature

Location:  
Payment API

Usage:  
TLS Authentication

Data Criticality:  
Critical

Quantum Vulnerability:  
High

Another:

Asset:  
AES-256

Location:  
Database Encryption

Usage:  
Data-at-rest

Quantum Vulnerability:  
Lower relative concern

Migration Priority:  
Low/Medium

This demonstrates why a simple algorithm inventory isn't enough.

---

## 35. Quantum Risk by Data Lifetime

Consider two applications.

### Application A

Data Lifetime:  
30 minutes

Crypto:  
RSA-2048

Migration:  
3 years

## Application B

Data Lifetime:  
30 years

Crypto:  
RSA-2048

Migration:  
3 years

The second system is substantially more concerning from a long-term confidentiality perspective.

Therefore, ECDAT should capture:

Data Retention Period

as part of its risk model.

---

## 36. Quantum Risk by Business Criticality

Not every application has the same business impact.

Example:

Marketing Website

versus:

National Security System

Both might use RSA-2048.

But their migration priorities are obviously different.

ECDAT should therefore support classifications such as:

LOW  
MEDIUM  
HIGH  
CRITICAL

or a more detailed enterprise taxonomy.

---

## 37. Quantum Risk by Attack Surface

A cryptographic asset may be:

### Internet-facing

Higher exposure.

### Internal

Potentially lower immediate exposure but still important.

### Air-gapped

Different threat model.

### Archived

Potential HNDL concern.

### Embedded

Potentially extremely difficult to migrate.

For example:

```
Legacy Embedded Device
  ↓
RSA-2048
  ↓
Firmware hard-coded
  ↓
No remote update
  ↓
Migration Complexity = VERY HIGH
```

This could be more operationally challenging than a modern web service.

---

## 38. Quantum Risk Categories

ECDAT could use multiple dimensions.

## Dimension 1 — Cryptographic Vulnerability

Safe  
Low  
Medium  
High  
Critical

## Dimension 2 — Data Sensitivity

Public  
Internal  
Confidential  
Restricted  
Highly Sensitive

## Dimension 3 — Data Lifetime

< 1 year  
1–5 years  
5–10 years  
10–20 years  
20+ years

## Dimension 4 — Migration Difficulty

Easy  
Moderate  
Difficult  
Very Difficult  
Unknown

## Dimension 5 — Business Criticality

Low  
Medium  
High  
Critical

This produces a much richer risk model.

---

## 39. Example Enterprise Scenario

Consider a fictional organization:

```
National Research Corporation
```

It operates:

```
5,000 applications
1,200 services
100,000 certificates
50,000 cryptographic keys
```

ECDAT scans the environment.

It discovers:

```
RSA
ECC
AES
SHA-256
SHA-1
3DES
```

The scanner creates an inventory.

---

## 40. Example ECDAT Analysis

Suppose ECDAT discovers:

```
Asset ID:
CRYPTO-000381

Algorithm:
ECDSA P-256

Location:
/payments/api/auth.py

Purpose:
API authentication

Business Criticality:
```

Critical

Data Lifetime:

15 years

Mosca calculation:

$15 + 3 > 12$

Therefore:

Risk:

CRITICAL

ECDAT might recommend:

Investigate migration to a standardized  
post-quantum signature mechanism.

Consider hybrid deployment during transition.

Validate compatibility with:

- TLS stack
- Client ecosystem
- HSM
- API gateway
- Certificate infrastructure

Notice that the recommendation is not simply:

"Replace ECDSA."

A production system must consider the entire dependency ecosystem.

---

## 41. What ECDAT Must Understand

A sophisticated ECDAT system should understand relationships.

For example:

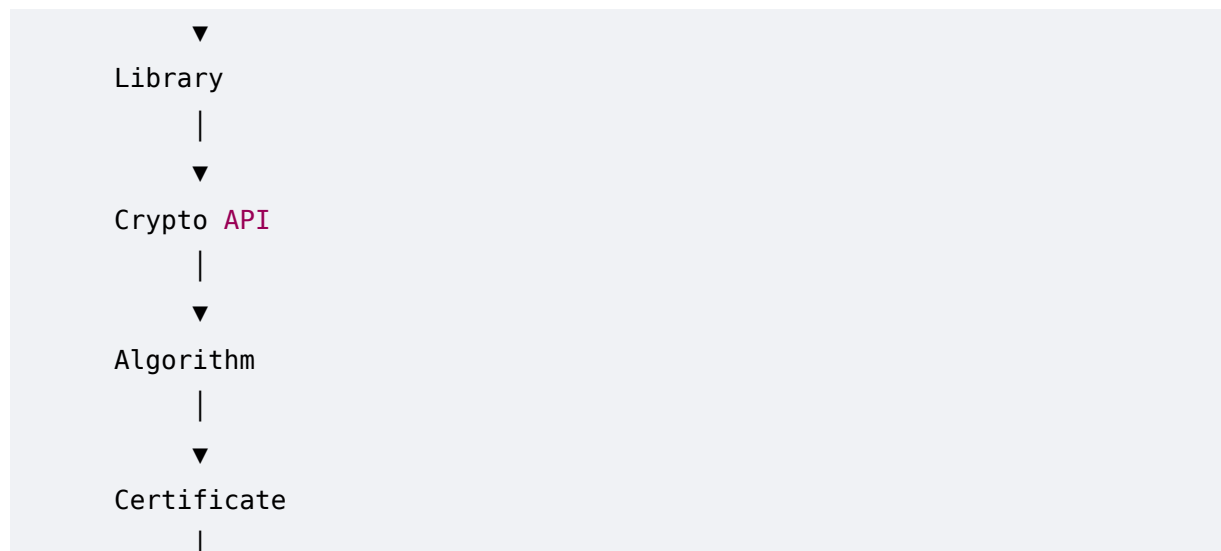
Repository

|



Application

|



This creates a dependency graph.

The important question becomes:

"Which business systems depend on quantum-vulnerable cryptography?"

That is much more valuable than:

"How many RSA references exist?"

---

## 42. Quantum-Aware Cryptographic Discovery

Traditional discovery:

```
Find RSA
Find ECC
Find AES
```

Quantum-aware discovery:

```
Find cryptographic asset
  ↓
Identify algorithm
  ↓
Identify exact variant
  ↓
Identify usage
  ↓
Identify key size
  ↓
Identify protocol
  ↓
```

```
Identify data protected
      ↓
Identify data lifetime
      ↓
Identify business criticality
```

This pipeline is the conceptual heart of ECDAT.

---

## 43. Quantum Risk Metadata

Every cryptographic asset could contain metadata such as:

```
Asset ID

Algorithm

Algorithm Family

Key Size

Protocol

Library

Library Version

Location

Application
```

This becomes part of the **Cryptographic Bill of Materials (CBOM)** discussed in later phases.

---

## 44. Quantum Risk Scoring

ECDAT should avoid relying solely on an LLM to determine risk.

Risk calculation should be deterministic wherever possible.

For example:

```
Risk Score =
Algorithm Risk
```



```
+
Data Sensitivity
+
Data Lifetime
+
Business Criticality
+
Migration Complexity
+
Exposure
```

A more sophisticated model could normalize these values.

For example:

```
Quantum Risk Score =
w1 × CryptoVulnerability
+
w2 × DataLifetime
+
w3 × BusinessCriticality
+
w4 × MigrationComplexity
+
w5 × Exposure
```

where:

```
w1 + w2 + w3 + w4 + w5 = 1
```

The exact weights should be configurable rather than arbitrarily fixed.

## 45. Quantum Risk Visualization

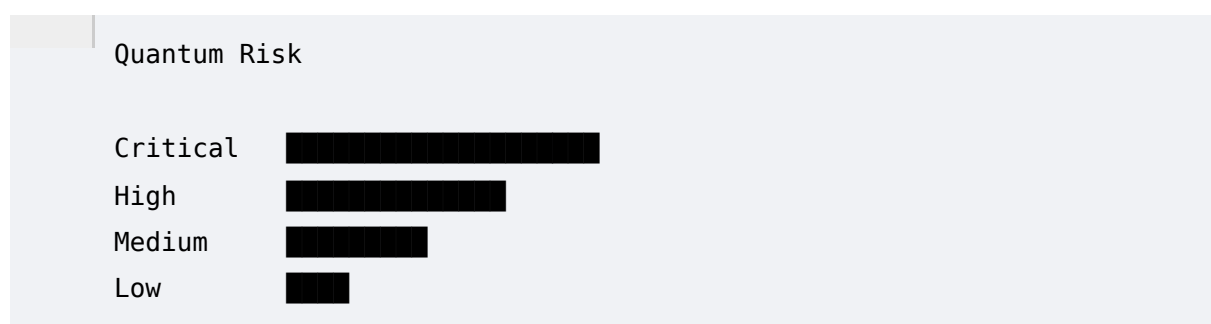
A dashboard could visualize risk using:

### Risk Matrix

|     | DATA LIFETIME |        |        |
|-----|---------------|--------|--------|
|     | Low           | Medium | High   |
| Low | Green         | Green  | Yellow |

|          |        |        |          |
|----------|--------|--------|----------|
| Medium   | Green  | Yellow | Orange   |
| CRYPTO   |        |        |          |
| RISK     | Yellow | Orange | Red      |
| High     |        |        |          |
|          | Orange | Red    | Critical |
| Critical |        |        |          |

## Another visualization



## 46. Quantum Migration Prioritization

ECDAT should answer:

"What should we migrate first?"

A possible priority model:

```

Priority =
Quantum Vulnerability
×
Business Criticality
×
Data Lifetime
×
Migration Difficulty

```

High-priority examples:

```

RSA
+
National-security data
+
30-year retention

```

+  
Complex migration

Low-priority examples:

Short-lived session  
+  
Non-sensitive data  
+  
AES-256

The system should produce an ordered migration backlog.

---

## 47. Common Misconceptions

### Misconception 1

"Quantum computers already break RSA."

Incorrect.

Current quantum systems are not equivalent to a cryptographically relevant fault-tolerant quantum computer.

---

### Misconception 2

"Everything becomes insecure because of quantum computing."

Incorrect.

The impact depends heavily on the cryptographic construction.

RSA/ECC face a fundamentally different threat from symmetric algorithms such as AES.

---

### Misconception 3

"AES-256 is broken by quantum computers."

Incorrect.

Grover's algorithm provides a quadratic generic speedup, not an instant break.

AES-256 retains a substantial security margin under the standard simplified analysis.

---

### Misconception 4

"We can migrate after quantum computers arrive."

Dangerous assumption.

Migration can take years or decades in large enterprises.

---

## Misconception 5

"Just replace RSA with ML-KEM."

Incorrect.

ML-KEM is a key-encapsulation mechanism.

It is not a direct drop-in replacement for every use of RSA.

Different cryptographic functions require different migration strategies.

---

## Misconception 6

"PQC migration only requires changing algorithms."

Incorrect.

Migration can affect:

- APIs
- Certificates
- HSMs
- Hardware
- Libraries
- Protocols
- Network appliances
- Embedded systems
- Compliance
- Performance
- Key management
- Software supply chains

This is why cryptographic inventory is essential.

---

## 48. Important Distinctions

ECDAT's documentation and AI system should maintain several distinctions.

## Quantum Vulnerable

Means the underlying cryptographic construction is expected to be broken by a sufficiently capable quantum computer.

## Currently Exploitable

Means there is a practical attack available today.

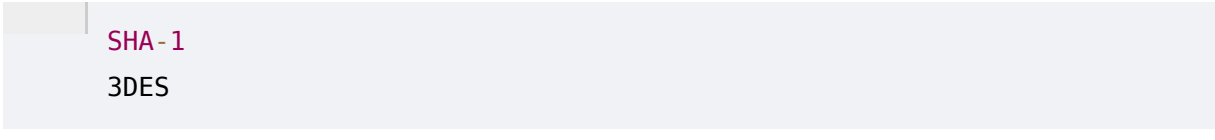
These are not the same.

---

## Deprecated

Means an algorithm or protocol is no longer recommended.

Example:



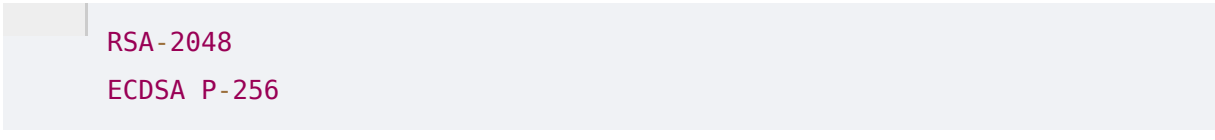
SHA-1  
3DES

---

## Quantum Vulnerable

May still be widely deployed and secure against known classical attacks.

Example:



RSA-2048  
ECDSA P-256

---

## Quantum Resistant

Means the construction is designed to resist known quantum attack strategies within its security model.

However, this does not mean:



"Mathematically guaranteed forever."

Cryptographic security always depends on assumptions and future cryptanalysis.

---

## 49. Engineering Implications for ECDAT

This phase produces several architectural requirements.

### Requirement 1

ECDAT needs a **cryptographic knowledge base**.

It should understand:

```
Algorithm
↓
Security Level
↓
Quantum Status
↓
Recommended Usage
↓
Migration Guidance
```

---

## Requirement 2

ECDAT needs a **quantum threat model**.

It should contain:

```
Threat Horizon
Attack Type
Algorithm Impact
Security Reduction
Confidence
```

---

## Requirement 3

ECDAT needs **data lifetime information**.

Without it, Mosca-style analysis becomes incomplete.

---

## Requirement 4

ECDAT needs **migration-time estimates**.

A system requiring 10 years to replace is very different from one that can be upgraded in 10 minutes.

---

## Requirement 5

ECDAT needs **business context**.

Purely technical analysis cannot determine organizational priority.

---

## Requirement 6

ECDAT needs **confidence scoring**.

Suppose the scanner discovers:

```
RSA.encrypt()
```

Confidence:

98%

But suppose AI infers:

```
CustomCryptoManager.encrypt()
```

and the implementation is ambiguous.

Confidence might be:

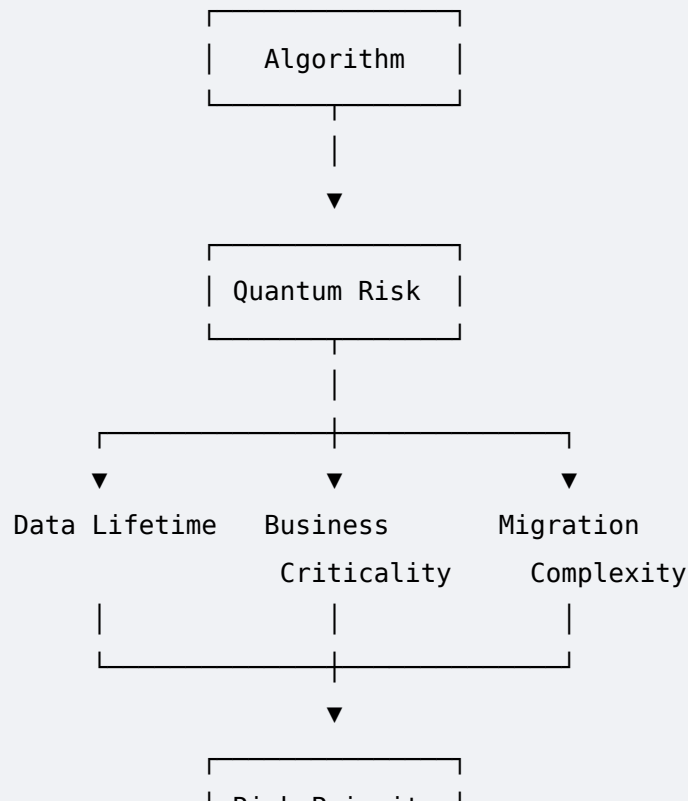
64%

This distinction is extremely important.

---

## 50. The ECDAT Quantum Knowledge Model

At the end of this phase, we can define the conceptual knowledge graph.

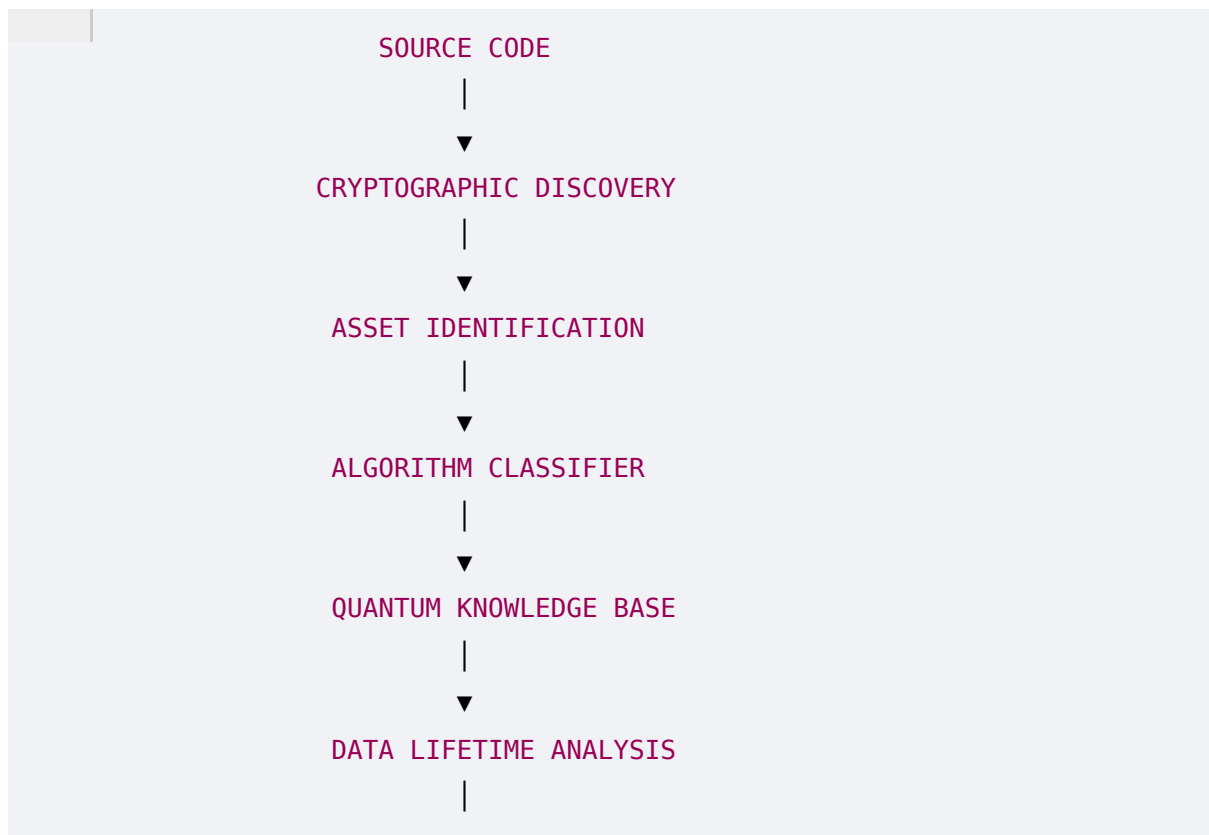


This is the foundation for the more advanced architecture we'll build later.

---

## 51. ECDAT's Quantum Risk Pipeline

The entire concept can be summarized as:



This pipeline should eventually become one of the central diagrams in your SIH presentation.

---

## 52. What Makes This Problem Hard

The difficulty of ECDAT becomes much clearer after understanding quantum computing.

The challenge is **not**:

"Can we detect RSA?"

That is relatively straightforward.

The real challenge is:

"Can we determine the organizational consequences of every cryptographic dependency in the context of an uncertain future quantum threat?"

That requires combining:



Cybersecurity

+

Cryptography

+

Quantum Computing

+

Software Engineering

+

Static Analysis

+

Cloud Security

+

AI

+

Graph Databases

+

Risk Management

This is why the problem is significantly more sophisticated than a normal SIH vulnerability scanner.

## 53. A Future ECDAT User Journey

Imagine a security administrator opening ECDAT.

The dashboard says:

ENTERPRISE QUANTUM READINESS

Cryptographic Assets: 184,231

Quantum-Vulnerable Assets: 71,492

Critical Assets: 4,281

High Priority Migrations: 12,402

Unknown Cryptographic Dependencies: 2,913

Overall PQC Readiness:

41%

The administrator clicks:

### Critical Quantum Risks

ECDAT displays:

1. National Identity Service

ECDSA P-256

Risk: CRITICAL

2. Secure Messaging Platform

RSA-2048

Risk: CRITICAL

3. Legacy VPN Infrastructure

RSA-2048 / DH

Risk: CRITICAL

4. Research Archive

ECC

Data Lifetime: 35 years

Risk: CRITICAL

The administrator then asks:

"Why is the Research Archive critical?"

The AI explains:

The archive contains information with an estimated 35-year confidentiality requirement.

The identified ECC dependency is vulnerable to future quantum attacks based on Shor's algorithm.

Estimated migration effort is 6 years.

Under the selected quantum threat scenario, the migration window is insufficient.

Recommendation:

Begin PQC migration planning immediately.

That is the kind of intelligence ECDAT should eventually deliver.

---

## 54. Phase 2 Key Concepts

At this point, the team should understand the following concepts thoroughly:

### Quantum Computing

- Qubits
- Superposition
- Measurement
- Entanglement
- Quantum gates
- Quantum circuits
- Quantum algorithms

### Quantum Cryptanalysis

- Shor's algorithm
- Grover's algorithm
- Integer factorization
- Discrete logarithms

### Cryptographic Impact

- RSA vulnerability
- DH vulnerability
- ECC vulnerability
- Digital signature vulnerability
- Symmetric cryptography impact
- Hash-function considerations

### Enterprise Risk

- CRQC
- HNDL
- Data lifetime
- Migration time
- Business criticality

- Mosca's inequality
- Quantum risk scoring

## ECDAT Requirements

- Quantum knowledge base
  - Algorithm classification
  - Quantum risk metadata
  - Migration estimation
  - Risk engine
  - Recommendation engine
  - Confidence scoring
- 

## 55. Phase 2 Summary

Quantum computing represents a fundamentally different threat to cryptography.

The important distinction is not:

"Quantum computers are fast."

The important distinction is:

**Certain quantum algorithms fundamentally change the computational difficulty of problems that modern cryptography relies upon.**

The most important example is **Shor's algorithm**, which threatens the mathematical foundations behind widely deployed public-key cryptography such as RSA and elliptic-curve systems.

**Grover's algorithm** creates a different, quadratic threat to generic search problems, resulting in a different impact on symmetric cryptography and hash functions.

The result is a major enterprise challenge.

Organizations cannot simply wait for a quantum computer to arrive.

They need to know:

What cryptography **do** we use?

Where is it used?

What data does it protect?

How long must that data remain secure?

How vulnerable is the cryptography?

How difficult is migration?

How long will migration take?

Which systems should migrate first?

What should replace the current cryptography?

That is exactly the gap ECDAT is designed to address.

---

## 56. The Core Insight

The central idea of this entire project can be expressed as:

**Quantum readiness begins with cryptographic visibility.**

You cannot migrate what you cannot find.

You cannot prioritize what you cannot understand.

You cannot assess quantum risk without understanding the cryptographic primitives involved.

And you cannot create a realistic migration roadmap without understanding the relationship between:

Cryptographic Asset

+

Data Lifetime

+

Business Criticality

+

Migration Complexity

+

Quantum Threat Horizon

ECDAT therefore starts with **discovery**, transforms discovery into **knowledge**, transforms knowledge into **risk**, and transforms risk into **action**.

---

## 57. Phase 3 Preview — Post-Quantum Cryptography

Now that we understand **why current cryptography is threatened**, the next question is:

### **What do we replace it with?**

Phase 3 will dive deeply into **Post-Quantum Cryptography (PQC)**.

It will cover:

- What PQC actually means
- Why "quantum-safe" is an imperfect term
- NIST's PQC standardization process
- ML-KEM
- ML-DSA
- SLH-DSA
- Lattice-based cryptography
- Hash-based cryptography
- Code-based cryptography
- Key encapsulation mechanisms
- Digital signatures
- Hybrid cryptography
- Crypto-agility
- PQC certificates
- TLS migration
- HSM migration
- Cloud migration
- Performance implications
- Key and signature sizes
- PQC deployment architecture
- Classical + PQC hybrid strategies
- How ECDAT should recommend PQC alternatives
- Algorithm-selection logic for the AI recommendation engine

**Phase 2 complete.**